

Roll No: BSSE23058

Name: Zunaira Abdul Aziz

## 1 Task 1

T1.

- (i) Implement a Gradio Image component configured to accept both live webcam input and image file uploads. When the student submits the form, the callback must receive the image as a NumPy array. Write the complete Gradio component definition and explain what the type='numpy' parameter means and why it is used here instead of type='filepath'.

```
# — Gradio UI layout (provided – do NOT remove components) —
with gr.Blocks(
    title="CampusConnect – FAST University SIS", theme=gr.themes.Soft()
) as app:

    state = gr.State(value=init_state())
    history_state = gr.State(value=[])

    gr.Markdown("# 🏠 CampusConnect – FAST University Support")
    gr.Markdown("***AI-powered multimodal student support chatbot**")

    with gr.Row():
        with gr.Column(scale=2):
            text_input = gr.Textbox(
                placeholder="Type your query here (5-500 chars)...",
                label="Text Input",
                lines=3,
                max_lines=5,
            )
            char_counter = gr.Textbox(
                label="Character Count", value="0 / 500", interactive=False
            )

            audio_input = gr.Audio(
                sources=["microphone", "upload"],
                type="filepath",
                label="Voice – Live Microphone or Audio Upload",
                info="Supports WAV and MP3",
            )

            transcript_box = gr.Textbox(
                label="Voice Transcript (auto-filled)", interactive=False, lines=2
            )

        with gr.Column(scale=2):
```

- (ii) Write a function process\_frame(image\_np) that: (a) checks if the input is None and returns a descriptive error string if so, (b) converts the frame from RGB (Gradio default) to BGR for OpenCV processing, (c) resizes it to a maximum of 640x480 while preserving aspect ratio using cv2.resize(), and (d) re-encodes it as a JPEG base64 string ready for the AI API. Show every step with the exact OpenCV and base64 calls used.

```

def process_frame(image_np) -> Optional[str]:
    """Task 1.A – receive NumPy frame, return base64 JPEG string or None"""

    try:
        if image_np is None:
            logger.warning("process_frame: image_np is None")
            return None

        image_bgr = cv2.cvtColor(image_np, cv2.COLOR_RGB2BGR)

        height, width = image_bgr.shape[:2]
        max_width, max_height = 640, 480

        if width > max_width or height > max_height:
            scale = min(max_width / width, max_height / height)
            new_width = int(width * scale)
            new_height = int(height * scale)
            image_bgr = cv2.resize(image_bgr, (new_width, new_height))

        success, jpeg_bytes = cv2.imencode(
            ".jpg", image_bgr, [cv2.IMWRITE_JPEG_QUALITY, 85]
        )
        if not success:
            logger.error("Failed to encode JPEG")
            return None

        base64_str = base64.b64encode(jpeg_bytes).decode("utf-8")
        logger.info(f"Frame processed: {len(base64_str)} bytes base64")
        return base64_str

    except cv2.error as e:
        logger.error(f"OpenCV error in process_frame: {e}")
        return None
    except Exception as e:
        logger.error(f"Error in process_frame: {e}")
        return None

```

- (iii) Implement a mode-toggle Radio button that switches between 'Live Webcam' and 'Upload Image/Video'. When 'Upload Image/Video' is selected, modify the Image component's sources parameter to ['upload'] only. When 'Live Webcam' is selected, restore sources to ['webcam']. In Gradio, explain how you dynamically update a component's properties after initial render using `gr.update()`.

```

        type="filepath",
        label="Voice – Live Microphone or Audio Upload",
        info="Supports WAV and MP3",
    )

    transcript_box = gr.Textbox(
        label="Voice Transcript (auto-filled)", interactive=False, lines=2
    )

    with gr.Column(scale=2):
        camera_input = gr.Image(
            sources=["webcam", "upload"],
            type="numpy",
            label="Camera – Live Webcam or Upload Image",
        )

    status_panel = gr.Textbox(label="Status", value="Ready", interactive=False)

```

- (iv) What happens when the user submits the form without providing any camera input (image\_np is None)? Write the guard clause that catches this case, prevents the API call from being made, and returns a gr.Warning() message visible to the user. Explain why checking for None before base64 encoding is critical and what error would occur without the check.

```

"""Task 2.B – main callback. Returns (chatbot, history, transcript, error_msg)"""

if state.get("is_processing", False):
    return (
        history,
        state,
        "",
        gr.Warning("Please wait for current response to complete."),
    )

state["is_processing"] = True
error_msg = ""
transcript = ""

```

T1.B

- (i) Implement a Gradio Audio component that accepts both live microphone recording and audio file upload (WAV and MP3). Write the transcribe\_audio(audio\_path) callback that: (a) checks if audio\_path is None and returns an appropriate message, (b) loads the Whisper 'base' model (cache it at module level — explain why), (c) calls model.transcribe() with the file path, and (d) returns the transcript string. Why must the model be loaded once at module level rather than inside the callback function?

```
def transcribe_audio(audio_path: str) -> str:
    """Task 1.B – transcribe WAV/MP3 using Whisper, return transcript string"""

    try:
        if audio_path is None:
            return ""

        audio_file = Path(audio_path)
        if not audio_file.exists():
            logger.error(f"Audio file not found: {audio_path}")
            return "Error: Audio file not found"

        if whisper_model is None:
            return "Error: Whisper not loaded. Cannot transcribe."

        logger.info(f"Transcribing: {audio_path}")
        result = whisper_model.transcribe(audio_path)
        transcript = result.get("text", "").strip()

        logger.info(f"Transcript: {transcript[:100]}...")
        return transcript

    except RuntimeError as e:
        if "ffmpeg" in str(e).lower():
            msg = "Error: MP3 requires ffmpeg. Please upload a WAV file instead or install ffmpeg."
            logger.error(msg)
            return msg
        else:
            logger.error(f"Whisper RuntimeError: {e}")
            return f"Error: {str(e)[:100]}"
    except Exception as e:
        logger.error(f"Error in transcribe_audio: {e}")
        return f"Error: Could not transcribe audio ({type(e).__name__})"
```

- (ii) The Gradio Audio component returns None when no audio is recorded. It also returns a file path to a temporary file when audio is provided. Write the complete conditional logic that handles both cases, validates that the file exists on disk before passing it to Whisper, and explains what FileNotFoundError scenario could still arise even after checking for None.

```
if isinstance(error, requests.exceptions.ConnectionError):
    return "Ollama server is not running. Please start Ollama and try again."
elif error_type == "APITimeoutError":
    return "The model is taking too long. Try a lighter model like mistral."
elif error_type == "APIConnectionError":
    return "Cannot reach local AI server at localhost:11434."
elif "UnknownValueError" in error_type:
    return "Could not understand audio. Please try again."
elif isinstance(error, FileNotFoundError):
    return "Audio or image file not found or corrupted."
elif isinstance(error, cv2.error):
    return "Camera error: check your device connection."
else:
    return f"Error: {str(error)[:80]}"
```

- (iii) After transcription completes, the transcript must be appended to the existing Textbox content (from Task 1.C) with a ' [voice] ' label prefix. Write the Gradio callback return

statement that appends without overwriting, given that both the transcript and the current textbox value are inputs to the function. Show the exact `gr.update()` call.

- (iv) A student uploads an MP3 file but Whisper raises a `RuntimeError` because `ffmpeg` is not installed. Write the complete error handling block that: (a) catches this specific scenario, (b) attempts to detect whether the error is `ffmpeg`-related by inspecting the exception message, (c) shows a user-friendly message explaining that MP3 conversion requires `ffmpeg` and suggesting they upload a WAV file instead, and (d) logs the full traceback to the console for debugging.

#### T1.C

- (i) Implement a Gradio Textbox with a maximum of 500 characters enforced server-side. Bind a function to the `.change()` event that: (a) returns a character count label (e.g., '312 / 500'), (b) returns `gr.update(value=text[:500])` if the text exceeds 500 characters to silently trim it, and (c) changes the counter label color to red (using Markdown) when fewer than 20 characters remain. Write all three behaviors in a single callback.

```
def update_char_counter(text: str) -> Tuple[str, str]:
    count = len(text)
    counter = f"{count} / 500"

    if count > 480:
        counter = f"{counter} (almost full!)"

    if count > 500:
        text = text[:500]

    return text, counter
```

- (ii) Gradio does not natively prevent copy-paste. Explain two strategies you can implement in Python/Gradio to detect and discourage pasted text: (a) comparing the change delta between two consecutive `.change()` events to flag unusually large input jumps, and (b) using a `gr.HTML()` component with a minimal JavaScript listener that disables the paste event on the specific textbox element. Show the implementation for both strategies.
- (iii) Write a `validate_text_input(text)` Python function that returns (`is_valid`: bool, `error_message`: str). It must reject: (a) empty or whitespace-only input, (b) input shorter than 5 characters, (c) input containing only numeric characters with no alphabetic content, and (d) input longer than 500 characters. The submit button's callback must call this function first and halt processing if `is_valid` is False, showing the `error_message` to the user.

```
def validate_text_input(text: str) -> Tuple[bool, str]:
    """Task 1.C – returns (is_valid: bool, error_message: str)"""

    if not text or text.isspace():
        return False, "Error: Text cannot be empty."

    if len(text) < 5:
        return False, "Error: Text must be at least 5 characters."

    if text.isdigit():
        return False, "Error: Text must contain letters, not only numbers."

    if len(text) > 500:
        return False, "Error: Text exceeds 500 character limit."

    return True, ""
```

## 2 Task 2

### T2.A

- (i) Write a `build_message_payload(text, transcript, image_b64)` Python function that constructs the messages list for one user turn in the OpenAI-compatible format used by Ollama. The function must: (a) always include the typed text as a text content block, (b) append the transcript as a separate labeled text block only if transcript is not empty, (c) prepend the base64 image block (type: `image_url`) only if `image_b64` is not `None`, and (d) return the content list in the order: image → transcript block → text block. Show the exact dict structure for each block type and the final messages list passed to `client.chat.completions.create()`.

```
def build_message_payload(
    text: str, transcript: str, image_b64: Optional[str]
) -> List[Dict]:
    """Task 2.A – returns OpenAI-compatible content list for one user turn"""

    content = []

    if image_b64 is not None:
        content.append(
            {
                "type": "image_url",
                "image_url": {"url": f"data:image/jpeg;base64,{image_b64}"},
            }
        )

    if transcript and transcript.strip():
        content.append({"type": "text", "text": f"[VOICE TRANSCRIPT]: {transcript}"})

    content.append({"type": "text", "text": text})

    return [{"role": "user", "content": content}]
```

- (ii) Write the system prompt string for CampusConnect. It must: (a) define the bot's persona and name, (b) explicitly list the five topic areas it may answer, (c) instruct it to refuse off-topic queries politely and name the correct office, (d) instruct it to always begin its response by acknowledging which input modality was most informative (e.g., 'Based on the image you shared...'), and (e) instruct it to respond in the same language the student used (Urdu or English). Write the full system prompt as a Python string.
- (iii) Implement a local `topic_classifier(text, transcript)` function that runs before any API call. It must use a keyword dictionary mapping each of the five topic areas to at least five keywords each. If neither the text nor the transcript matches any topic, the function returns `(False, 'off-topic message string')`. If a match is found, it returns `(True, topic_name)`. Show the full keyword dict and the matching logic. Explain why running this locally saves API tokens.
- (iv) The API context window has a token limit. Implement a `trim_history(history, system_prompt, max_turns=10)` function that: (a) always keeps the system prompt at position 0, (b) keeps only the last `max_turns` message dicts, and (c) logs a warning to the console when trimming occurs. Show where in the main chat function you call `trim_history` before passing history to the API.

## T2.B

- (i) Write the complete `chat(text, audio_path, image_np, history, session_id)` Gradio callback function. It must: (a) validate all inputs using Task 1 validators, (b) process image via `process_frame()`, (c) transcribe audio via `transcribe_audio()`, (d) classify topic, (e) build the full messages list using `build_message_payload()`, (f) call the Ollama API via the OpenAI-compatible client at `http://localhost:11434/v1`, (g) append both user and assistant turns to history, and (h) return the updated history to the Chatbot component and updated `gr.State`. Show the complete function signature and body including the client initialization.

```

def chat(text: str, audio_path: Optional[str], image_np: Optional[np.ndarray], history: List[Dict])
    """Task 2.B – main callback. Returns (chatbot, history, transcript, error_msg)"""

    if state.get("is_processing", False):
        return (
            history,
            state,
            "",
            gr.Warning("Please wait for current response to complete."),
        )

    state["is_processing"] = True
    error_msg = ""
    transcript = ""

    try:
        is_valid, validation_error = validate_text_input(text)
        if not is_valid:
            state["is_processing"] = False
            return history, state, "", validation_error

        image_b64 = process_frame(image_np) if image_np is not None else None
        transcript = transcribe_audio(audio_path) if audio_path else ""

        in_scope, topic_or_msg = topic_classifier(text, transcript)

        if not in_scope:
            user_msg = f"Text: {text}"
            if transcript:
                user_msg += f"\n[Voice: {transcript}]"

            history.append({"role": "user", "content": user_msg})
            history.append({"role": "assistant", "content": topic_or_msg})

            state["is_processing"] = False
            return history, state, transcript, ""

```

Ln 413, Col 17 Spaces: 4 UTF-8 CRLF { } D



```

user_messages = build_message_payload(text, transcript, image_b64)

system_msg = {"role": "system", "content": SYSTEM_PROMPT}
full_history = [system_msg] + history + user_messages

full_history = trim_history(full_history, max_turns=10)

logger.info(f"Calling Ollama with {len(full_history)} messages")

response_text = ""
try:
    stream = client.chat.completions.create(
        model="llama3",
        messages=full_history,
        stream=True,
        temperature=0.7,
        top_p=0.9,
    )

    for chunk in stream:
        if chunk.choices[0].delta.content:
            response_text += chunk.choices[0].delta.content

except Exception as e:
    response_text = handle_error(e, "Ollama API call")
    error_msg = response_text

if not response_text.startswith("X") and not response_text.startswith(
    "Warning"
):
    history.append({"role": "user", "content": text})
    history.append({"role": "assistant", "content": response_text})

state["conversation_history"] = history
state["last_error"] = error_msg if error_msg else None

```

- (ii) Implement streaming for the Ollama response. Pass `stream=True` to `client.chat.completions.create()`. Write a Python generator function that iterates over the streaming response chunks, extracts `chunk.choices[0].delta.content`, and yields the progressively accumulated response string. Explain how Gradio's generator-based streaming works: what must the function use instead of `return`, and how does Gradio know to update the Chatbot component on each yield?
- (iii) Implement a 'Copy Response' button next to each AI message. Since Gradio does not natively support per-bubble copy buttons, implement this by adding a `gr.HTML()` component that renders a small copy icon using JavaScript's `navigator.clipboard.writeText()`. The text to copy must be passed from Python to the HTML component via `gr.update(value=html_string)` on each new response. Show the complete implementation.
- (iv) Implement a 'Clear Conversation' button that: (a) resets the history `gr.State` to an empty list, (b) clears the Chatbot display, (c) clears the Textbox, (d) resets the Image component to `None`, (e) resets the Audio component to `None`, and (f) generates a new

session\_id in the session State. Write the clear\_all() function with the correct tuple of return values matching the outputs list of the button's click event.

```
def clear_all(state: Dict) -> Tuple[List, List, str, str, str, None, None, Dict]:
    state["conversation_history"] = []
    state["session_id"] = str(uuid.uuid4())

    return (
        [], # chatbot
        [], # history state
        "", # text input
        "0 / 500", # char counter
        "", # transcript
        None, # image
        None, # audio
        state, # state
    )
```

- (v) After the AI responds, parse the response text to extract which modality it claims to have used (look for phrases like 'Based on the image', 'From your spoken query', 'From your typed message'). Write a parse\_modality\_used(response\_text) function that returns one of: 'image', 'voice', 'text', or 'combined'. Use this result to update a gr.Textbox label in the UI showing 'Primary modality understood: image' so the student always knows how the bot interpreted their input.

```
def parse_modality_used(response_text: str) -> str:
    response_lower = response_text.lower()

    if any(
        phrase in response_lower
        for phrase in [
            "based on the image",
            "from the document",
            "from your screenshot",
        ]
    ):
        return "image"
    elif any(
        phrase in response_lower
        for phrase in ["from your spoken", "your voice", "audio you"]
    ):
        return "voice"
    elif any(
        phrase in response_lower
        for phrase in ["from your typed", "from your message", "question you asked"]
    ):
        return "text"
    else:
        return "combined"
```

- (i) Define the complete initial application state as a Python dictionary stored in a `gr.State()` component. It must include: `session_id` (uuid), `conversation_history` (list), `is_processing` (bool), `last_error` (str or None), `camera_mode` ('webcam' or 'upload'), `voice_mode` ('mic' or 'upload'), and `char_count` (int). Explain why `is_processing` must be part of State rather than a module-level global when multiple students use the app simultaneously.
- (ii) Write an `update_state(current_state, **kwargs)` helper function that merges keyword argument updates into the state dict and returns the new dict. Demonstrate its use: show how a single line in the submit callback can set `is_processing=True` and update `last_error=None` simultaneously. Explain why always returning a new dict (rather than mutating in place) is safer in a concurrent Gradio environment.

```
def update_status_panel(state: Dict) -> str:
    is_processing = state.get("is_processing", False)
    status = "Processing..." if is_processing else "Ready"

    return f"Status: {status}"
```

- (iii) Implement a submit guard: at the very start of the `chat()` callback, check `state['is_processing']`. If True, immediately return all current outputs unchanged plus a `gr.Warning('Please wait for the current response to complete.')`. Set `is_processing=True` in the state before the API call and False in a finally block. Write the complete try/finally structure that guarantees `is_processing` is reset even if an exception occurs mid-call.
- (iv) When the user switches `camera_mode` from 'webcam' to 'upload' via a Radio button, the Image component must update its sources. Write the `mode_switch(selection, state)` callback that: (a) updates `camera_mode` in state, (b) returns `gr.update(sources=['webcam'])` or `gr.update(sources=['upload'])` based on selection, and (c) also resets the current image value to None so the old capture is not accidentally submitted with the new mode. Show the full callback and its Gradio event binding.

### T3.B

- (i) Write a unified `handle_error(error: Exception, context: str)` Python function that: (a) logs the full traceback using `logging.exception()`, (b) maps known exception types to user-friendly messages: `requests.exceptions.ConnectionError` → 'Ollama server is not running. Please start Ollama and try again.', `openai.APITimeoutError` → 'The model is taking too long to respond. Try a lighter model like mistral.', `openai.APIConnectionError` → 'Cannot reach the local AI server at localhost:11434.', `sr.UnknownValueError` → 'Could not understand audio, please try again.', `FileNotFoundError` → 'Audio or image file not found or corrupted.', `cv2.error` → 'Camera error: check your device connection.', and (c) returns a safe default string for any other Exception. Show the complete function with all branches.

```

def handle_error(error: Exception, context: str = "") -> str:
    """Task 3.B – unified error handler, returns user-friendly string"""

    logger.exception(f"Error in {context}: {error}")

    error_type = type(error).__name__
    error_msg = str(error).lower()

    if isinstance(error, requests.exceptions.ConnectionError):
        return "Ollama server is not running. Please start Ollama and try again."
    elif error_type == "APITimeoutError":
        return "The model is taking too long. Try a lighter model like mistral."
    elif error_type == "APIConnectionError":
        return "Cannot reach local AI server at localhost:11434."
    elif "UnknownValueError" in error_type:
        return "Could not understand audio. Please try again."
    elif isinstance(error, FileNotFoundError):
        return "Audio or image file not found or corrupted."
    elif isinstance(error, cv2.error):
        return "Camera error: check your device connection."
    else:
        return f"Error: {str(error)[:80]}"

```

- (ii) Implement an exponential backoff retry decorator `@retry_with_backoff(max_retries=3, base_delay=2)` for the Ollama API call. The decorator must: (a) retry only on Human Computer Interaction Dr. M. Asif `openai.APIConnectionError` and `openai.APITimeoutError` (both mean the server is busy or slow, not a permanent error), (b) wait `base_delay ** attempt` seconds between retries, (c) update a `gr.Textbox` label with 'Retrying in Xs...' countdown before each wait, and (d) raise the final exception after `max_retries` are exhausted. Write the complete decorator.

```

def retry_with_backoff(max_retries: int = 3, base_delay: float = 2):
    def decorator(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            for attempt in range(max_retries):
                try:
                    return func(*args, **kwargs)
                except Exception as e:
                    error_type = type(e).__name__
                    if error_type not in ["APIConnectionError", "APITimeoutError"]:
                        raise

                    if attempt == max_retries - 1:
                        raise

                    delay = base_delay**attempt
                    logger.warning(
                        f"Attempt {attempt + 1} failed. Retrying in {delay}s..."
                    )
                    time.sleep(delay)

            return wrapper

    return decorator

```

- (iii) On application startup (before `gr.Blocks()` launches), perform a system health check: (a) verify Ollama is running by sending GET `http://localhost:11434/api/tags` and checking for a 200 response, (b) verify the 'llama3' model is available by checking that 'llama3' appears in the tags response JSON, (c) verify Whisper can be imported and the 'base' model file exists in cache, (d) verify OpenCV can open camera index 0. Print a color-coded PASS/FAIL report to the console for each check. If Ollama or llama3 is not found, print a clear instruction ('Run: `ollama pull llama3`') and raise a `RuntimeError` before the app starts.

### T3.C

- (i) Implement a status indicator panel using a `gr.Dataframe` or three `gr.Textbox` components (one per modality) that always shows the current status of camera (Active / Off / Error), voice (Transcribing / Ready / Error), and text (X/500 chars). This panel must update after every user action. Write the `update_status_panel(state)` function that reads from the state

dict and returns the three status strings.

```
def update_status_panel(state: Dict) -> str:
    is_processing = state.get("is_processing", False)
    status = "Processing..." if is_processing else "Ready"

    return f"Status: {status}"

# — Gradio UI layout (provided – do NOT remove components) —
with gr.Blocks(
    title="CampusConnect – FAST University SIS", theme=gr.themes.Soft()
) as app:
```

- (ii) Gradio automatically generates some accessibility attributes, but you must ensure: (a) every `gr.Button` has a `descriptive_label=` parameter (not just an icon), (b) every `gr.Image` and `gr.Audio` has a `label=` parameter that clearly describes its purpose, and (c) the `gr.Chatbot` component has `show_label=True` with a meaningful label. Rewrite the component definitions from your Task 1 implementation to include all required accessibility labels.
- (iii) Implement a 'dark mode / light mode' toggle using Gradio's built-in theme system. Add a `gr.Button('Toggle Theme')` that switches between `gr.themes.Default()` and `gr.themes.Monochrome()`. Since Gradio themes are set at app launch, explain the alternative approach: use a `gr.HTML()` component to inject a CSS class toggle on the root element via JavaScript, and provide the minimal CSS needed to invert background and text colors. Show both the Python Gradio approach and the JavaScript fallback.

```
status = "Processing..." if is_processing else "Ready"

    return f"Status: {status}"

# — Gradio UI layout (provided – do NOT remove components) —
with gr.Blocks(
    title="CampusConnect – FAST University SIS", theme=gr.themes.Soft()
) as app:

    state = gr.State(value=init_state())
    history_state = gr.State(value=[])

    gr.Markdown("# 🏠 CampusConnect – FAST University Support")
    gr.Markdown("***AI-powered multimodal student support chatbot***")
```

#### 4 Task 4

- (i) Design and execute all EIGHT required test cases below. For each, document in a table: Test ID, Precondition, Steps Executed, Expected Result, Actual Result, Pass / Fail. TC-01: Submit with only typed text (no audio, no image). TC-02: Submit with only a webcam-captured image (no text, no audio). TC-03: Submit with all three modalities simultaneously. TC-04: Submit with an empty Textbox while audio and image are provided. TC-05: Upload an MP3 audio file and verify transcription or graceful error. TC-06: Submit while `is_processing` is True (rapid double-click simulation). TC-07: Type exactly 501 characters and verify the 500-char

limit is enforced. TC-08: Switch from webcam to image upload mid-session and verify the previous capture is cleared.

- (ii) Design and execute FOUR additional edge-case test cases of your own choosing. Each must target a different failure mode not covered by TC-01 to TC-08 (examples: Ollama server stopped mid-session, Whisper model file deleted after app start, camera disconnected mid-session, history exceeding 10 turns triggering the sliding window trim). Document in the same table format.
- (iii) For every test case that received a FAIL result: write (a) the Root Cause — one sentence identifying the exact function name, line number or logic, and reason for failure, and (b) the Proposed Fix — one sentence or a code snippet of max 5 lines showing the correction. Do not leave any FAIL row without analysis. A blank analysis for a FAIL earns zero for that row.

#### T4.B

- (i) Write a structured one-page test summary report containing: (a) Total test cases designed, Pass count, Fail count, Partial count. (b) The single most critical unresolved defect: its Test ID, description, severity (Critical / Major / Minor), and the direct user impact in one sentence. (c) One concrete recommendation for the team before production deployment. (d) Your overall system confidence rating (1–5) with a two-sentence justification.
- (ii) Identify the single most significant HCI design weakness in your implementation — not a code bug, but a design decision that reduces usability for the target user (FAST University students). State: (a) HCI principle is violated, (b) the exact element in your UI that violates it, (c) a specific, implementable design change that would fix it, and (d) how you would verify the fix improved usability (one-sentence test method).